گزارش پروژه

فلوچارت پروژه جدول نمادها

Symbol Table

زبان برنامه نویسی	C++
ساختار داده	جدول هش با ۳ خانه و لیست پیوندی در هر خانه
تهیه کننده	زینب منفردی

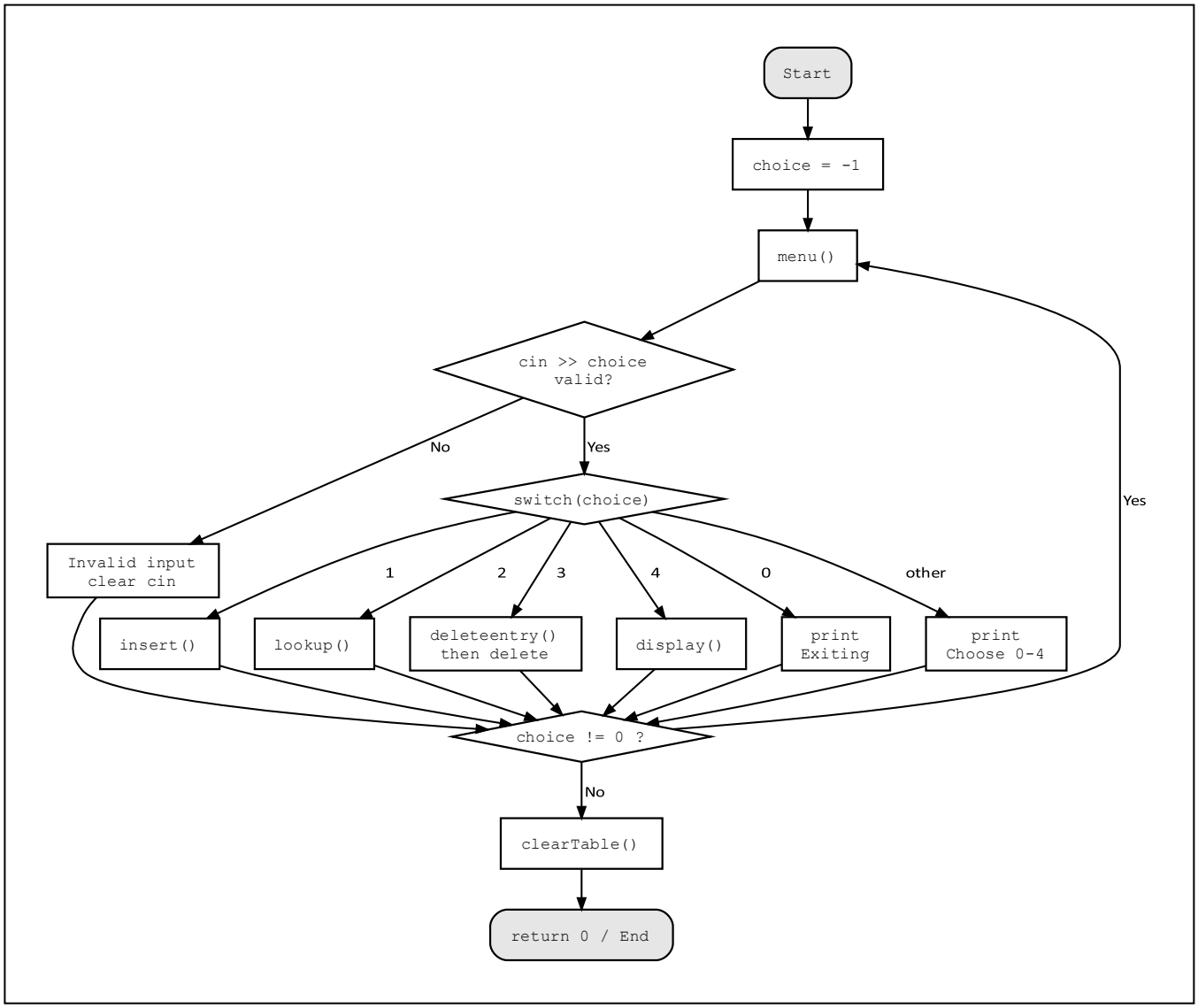
فهرست فلوچارت ها

ردیف	عنوان	تابع	صفحه
۱	برنامه اصلی	`main`	۲
۲	افزودن نماد	`insert`	۳
۳	جستجو	`lookup / searchoneline`	۴
۴	حذف	`deleteentry`	۵
۵	نمایش جدول	`display`	۶
۶	آزادسازی حافظه	`clearTable`	۷

راهنمای نمادها

۱ برنامه اصلی `main`

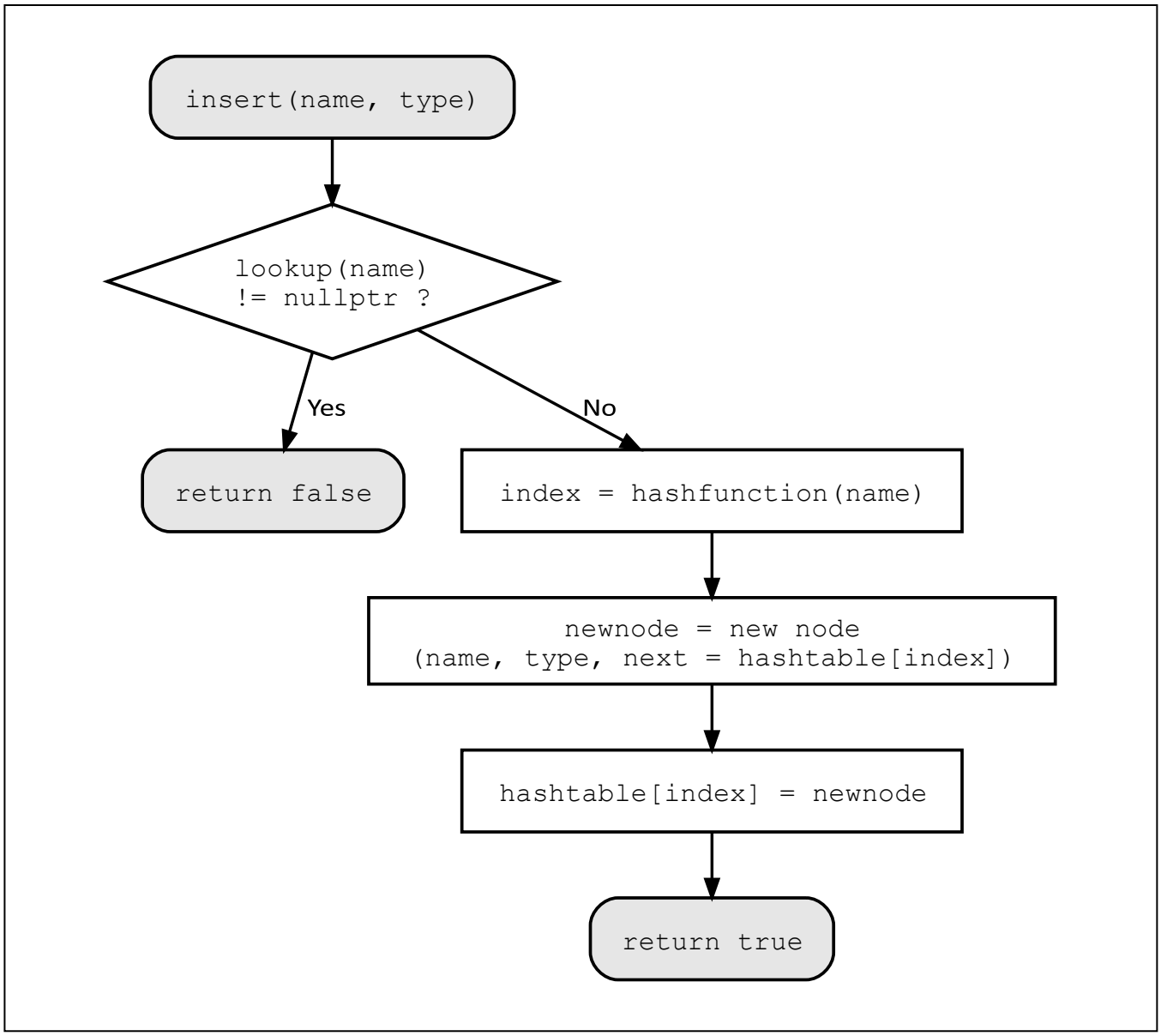
منو را نمایش می دهد، انتخاب کاربر را می خواند و تابع مربوطه را اجرا می کند. این روند تا زمانی که کاربر عدد ۰ را وارد کندتکرار می شود.



Code	شرح مرحله
`menu(); cin >> choice`	نمایش منو و خواندن شماره انتخاب
`cin.clear(); cin.ignore(10000, '\n')`	در صورت ورودی نامعتبر، پیام خطا چاپ و ورودی پاک می شود
`switch (choice)`	اجرای یکی از چهار عملیات بر اساس شماره انتخاب
`while (choice != 0)`	تکرار منو تا زمانی که شماره انتخاب صفر نباشد
`clearTable()`	آزادسازی تمام گره ها پیش از پایان برنامه

۲ افزودن نماد `insert`

در صورت تکراری نبودن نام، گره جدید را در ابتدای لیست خانه مربوطه قرار م یدهد و مقدار true برمی گرداند.

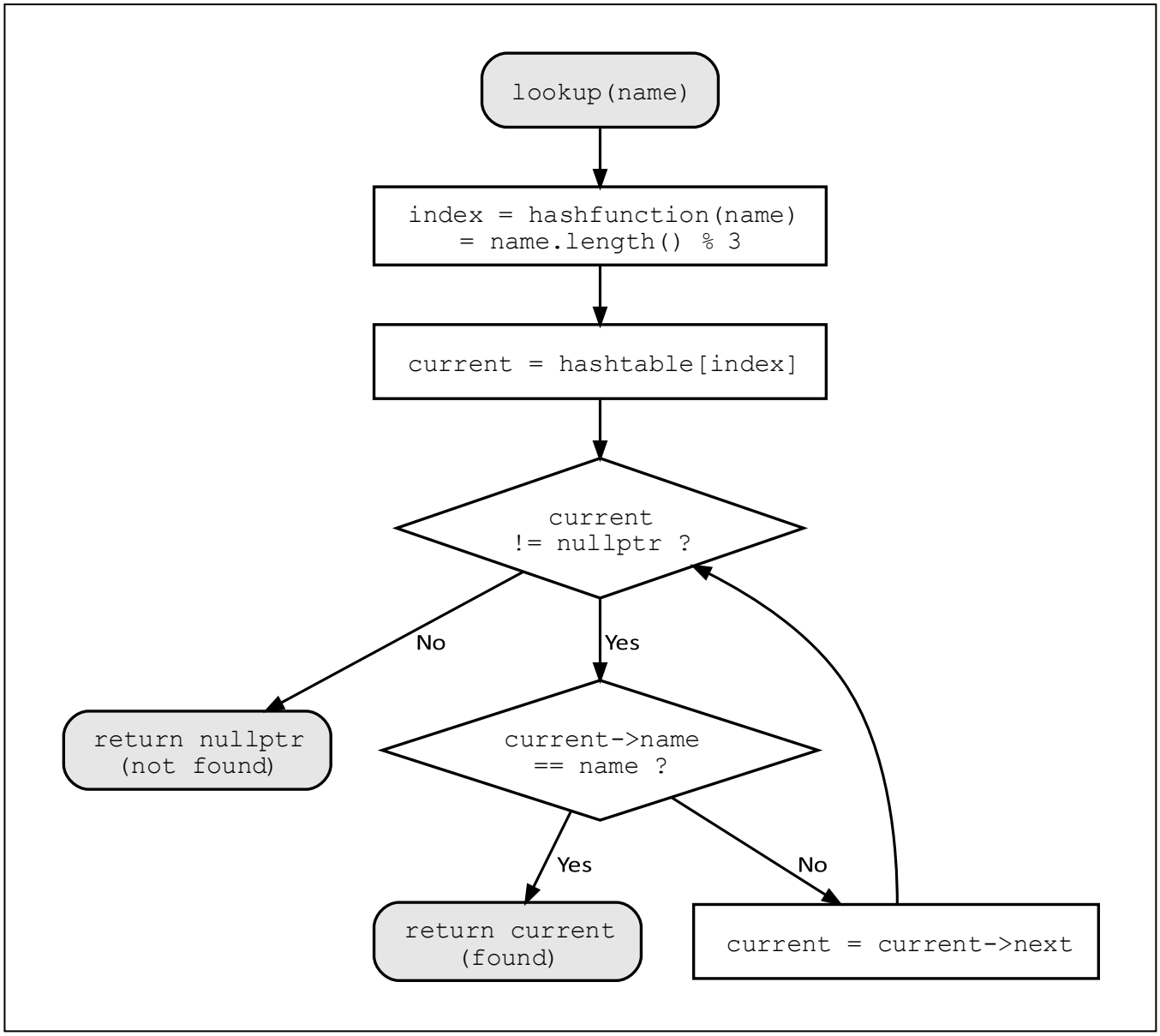


شرح مرحله	**Code**
بررسی وجود نام؛ در صورت وجود، false بازگردانده می شود	`lookup(name) != nullptr`
محاسبه شماره خانه جدول	`hashfunction(name)`
ساخت گره جدید که next آن به سر فعلی لیست اشاره می کند	`new node{name, type, hashtable[index]}`
گره جدید سر لیست می شود	`hashtable[index] = newnode`

۳جستجو

`lookup / searchoneline`

تابع lookup با تابع هش خانه مربوطه را پیدا می کند و تابع searchoneline لیست پیوندی آن خانه را گره به گرهپیمایش می کند.



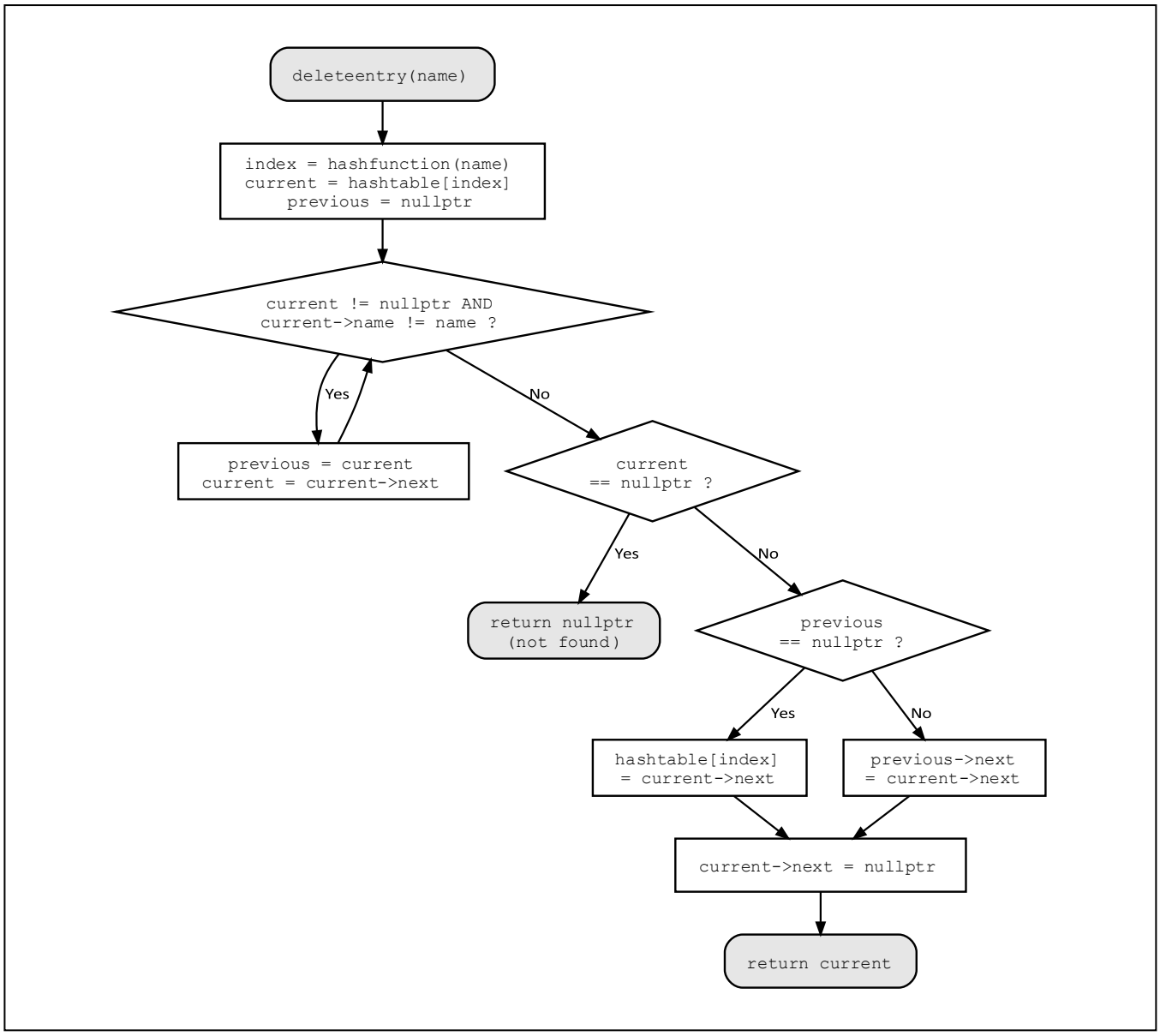
Code	شرح مرحله
`name.length() % tablesize`	شماره خانه برابر باقی مانده طول نام بر اندازه جدول است
`current = hashtable[index]`	آدرس اولین گره خانه در current قرار می گیرد
`current != nullptr`	اگر current تهی باشد، به انتهای لیست رسیده ایم و نام یافت نشده است
`current->name == name`	اگر نام گره با نام جستجو برابر باشد، آدرس همان گره بازگردانده می شود

در غیر این صورت به گره بعدی می رویم	`current = current->next`

۴حذف

`deleteentry`

گره موردنظر را پیدا می کند و از لیست جدا می کند. آزادسازی حافظه (delete) در تابع main انجام می شود.

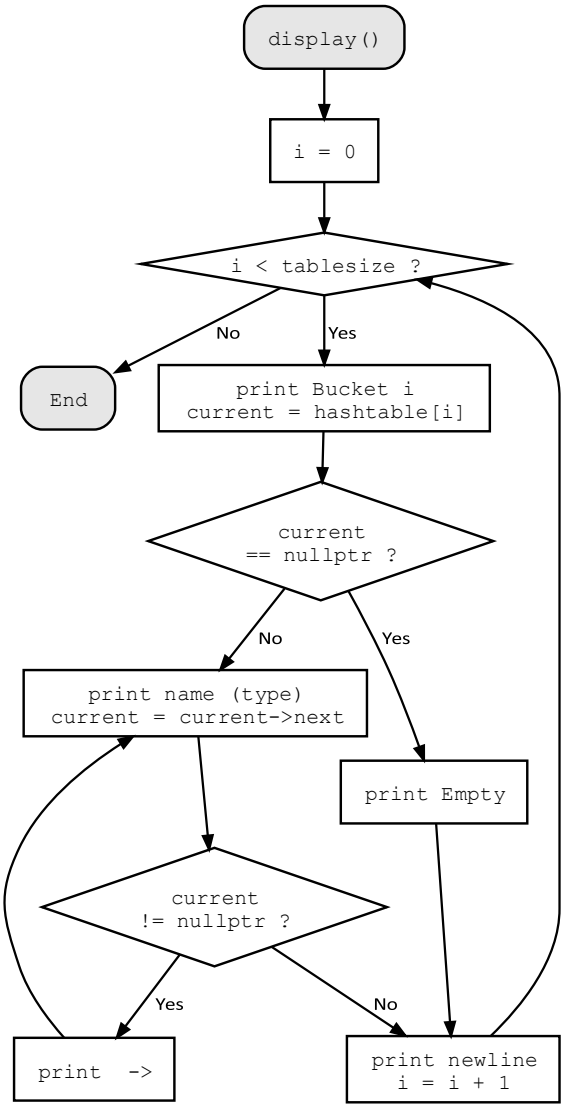


شرح مرحله	**Code**
پیمایش لیست با نگهداری گره قبلی	`previous = current`
در صورت یافت نشدن نام، nullptr بازگردانده می شود	`current == nullptr`
اگر گره سر لیست باشد، سر لیست به گره بعدی منتقل می شود	`hashtable[index] = current->next`

اگر گره وسط یا انتهای لیست باشد، گره قبلی به گره بعد از آن وصلمی شود	`previous->next = current->next`
گره جداشده بازگردانده می شود تا main آن را آزاد کند	`return current`

۵ نمایش جدول `display`

تمام خان ههای جدول را به ترتیب چاپ می کند و زنجیره گره های هر خانه را نشان می دهد.

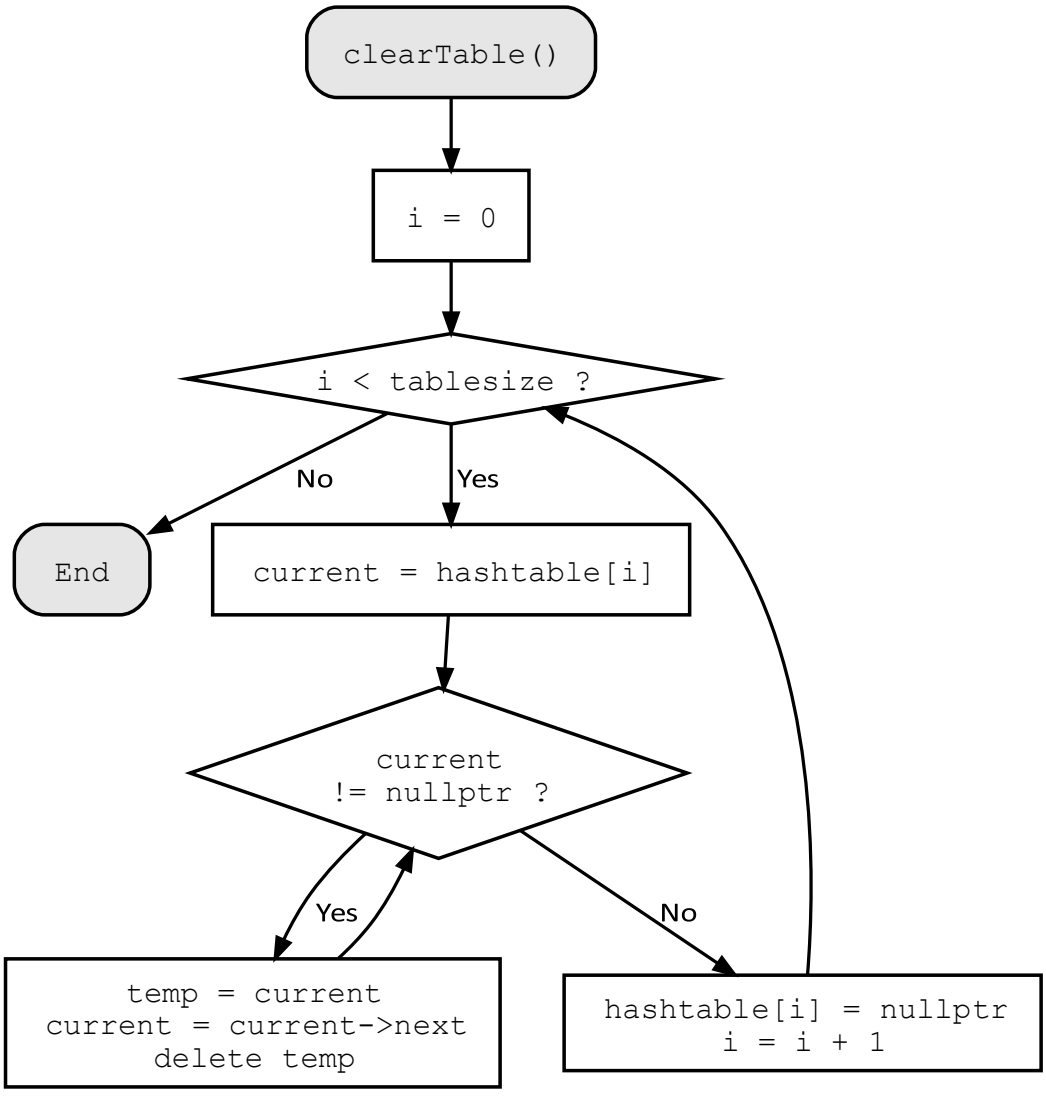


شرح مرحله	**Code**
حلقه روی شماره خانه ها	`for (i = 0; i < tablesize; i++)`
در صورت خالی بودن خانه، عبارت Empty چاپ می شود	`cout << "Empty"`

نام و نوع هر گره چاپ می شود و بین گره ها فلش قرار می گیرد	`name (type) -> name (type)`

۶ آزادسازی حافظه `clearTable`

تمام گر ههایی که با new ساخته شده اند را با delete آزاد می کند تا نشت حافظه رخ ندهد.



شرح مرحله	**Code**
آدرس گره فعلی پیش از حرکت در temp ذخیره می شود	`temp = current`
حرکت به گره بعدی و سپس آزادسازی گره قبلی	`current = current->next; delete temp`
پس از خالی شدن لیست، خانه جدول به nullptr بازمی گردد	`hashtable[i] = nullptr`